

Group E2EE Ephemeral Messaging Plan

This document outlines the architecture, cryptographic systems, and implementation modifications to establish a secure, receipt-based Group E2EE Ephemeral Messaging system in Qbase. Under this framework, messages are securely encrypted end-to-end and are dynamically cleaned up from the Appwrite cloud relay the exact second all group participants have successfully received and written them to their local Room database.

1. Underlying Cryptographic Architecture

Qbase uses Google Tink to implement hybrid asymmetric encryption:

- **Symmetric Encryption:** The message payload is encrypted once using an AES-128-GCM session key.
- **Asymmetric Key Wrapping:** The session key is encrypted (wrapped) for every participant in the group using their individual public keys.
- **Serialization:** The wrapped keys are stored inside the `wrappedKey` JSON string attribute of the message:
`{"userId1": "wrappedKey1", "userId2": "wrappedKey2"}`

2. Receipt-Based Ephemeral Relay Protocol

To allow offline group members to receive messages, we use a decentralized receipt-based erasure protocol:

1. **Upload:** Senders upload messages with key wraps for all recipients.
2. **Download & Decrypt:** Receivers fetch, decrypt, and save the message locally.
3. **Acknowledgment:**
 - The receiver parses the `wrappedKey` JSON map and deletes their own entry.
 - If other recipients (excluding the sender) are still present in the map, the receiver updates the document on Appwrite with the shrunk map.
 - If no other recipients remain, the last receiver completely deletes the document from the cloud.

3. Implemented Integration Method

We introduce a unified delivery acknowledgment method inside `SyncRepository.kt`:

```
private suspend fun acknowledgeMessageDelivery(  
    messageId: String,  
    chatId: String,  
    senderId: String,
```

```

        wrappedKeyStr: String
    ) {
        if (senderId == currentUserId) return

        val chat = chatDao.getChatById(chatId) ?: return
        if (!chat.isGroup) {
            try {
                databases.deleteDocument("qbase_db", "messages", messageId)
            } catch (e: Exception) {
                Log.e("SyncRepository", "P2P Ephemeral failed", e)
            }
            return
        }

        try {
            val wrappedKeysMap = deserializeWrappedKeys(wrappedKeyStr).toMutableMap()
            wrappedKeysMap.remove(currentUserId)

            val pendingReceivers = wrappedKeysMap.keys.filter { it != senderId }

            if (pendingReceivers.isEmpty()) {
                databases.deleteDocument("qbase_db", "messages", messageId)
            } else {
                val updatedWrappedKeyStr = serializeWrappedKeys(wrappedKeysMap)
                databases.updateDocument(
                    databaseId = "qbase_db",
                    collectionId = "messages",
                    documentId = messageId,
                    data = mapOf("wrappedKey" to updatedWrappedKeyStr)
                )
            }
        } catch (e: Exception) {
            Log.e("SyncRepository", "Group Ephemeral failed", e)
        }
    }
}

```

This single helper integrates across the initial offline sync block, active chat Realtime stream, and background Realtime stream.